Name: P H Devesh
Register Number: 1911260236
Subject: Python Programming (CSA0805)
Assignment: CO1

TASK 1 – Understand the Problem and Identify the Data

Problem analysis

The application records one milk delivery for a farmer in a particular shift and calculates the price per litre and the payment. The standard quality is 3.5% fat and 8.5% SNF, for which the base rate is Rs 30.00 per litre. Each 0.1 percentage-point fat difference changes the rate by Rs 0.40; each 0.1 percentage-point SNF difference changes it by Rs 0.30. A farmer's ten-day total is the sum of the payments for that farmer during the ten-day period.

Inputs and outputs

Inputs: farmer_id (text, because it may contain letters or leading zeroes), litres (fractional numeric value), fat_percent and snf_percent (fractional numeric values), and shift (text such as Morning or Evening). Outputs: rate_per_litre in rupees per litre, payment for the entry in rupees, and the farmer's ten-day total. The date or payment-period identifier should also be stored in a production system so that entries are grouped into the correct ten-day cycle.

Constants, types and validation

The base rate Rs 30.00, reference fat 3.5, reference SNF 8.5, fat adjustment Rs 0.40 per step, SNF adjustment Rs 0.30 per step, step size 0.1, and allowed fat limits 2.0 and 7.0 are configuration constants. Farmer ID and shift are strings; litres, fat and SNF are fractional values; step counts are integers; validity checks produce booleans; and a collection of entries can be stored in a list of tuples or records. Quantity must be greater than zero. Fat must be between 2.0% and 7.0%, inclusive. SNF must be within the range configured by the co-operative; the assignment does not state an SNF range, so it must be confirmed rather than invented. Inputs must be numeric and finite. A valid shift and farmer ID are required.

Assumptions

The 0.1% step means a difference of 0.1 percentage point, not a relative percentage. Positive differences increase the rate and negative differences decrease it. The base rate is applied to each litre. The rate is rounded to two decimal places and the entry payment is rounded to two decimal places before being included in the ten-day payment, unless the co-operative's bank reconciliation policy specifies rounding only at settlement. The chosen rounding rule must be applied consistently. A farmer can have multiple entries per day and across shifts.

Alternative concept and complexity

With separate variables such as litres1, fat1, snf1, litres2, fat2, snf2, and so on, the program would need manually repeated calculations and separate names for every farmer. For n entries this is still O(n) time if every entry is processed, but the code size and maintenance effort grow with n; it is not a practical data model. A list containing one tuple per entry, for example (farmer_id, litres, fat, snf, shift), uses O(n) storage and supports a single loop that processes all entries in O(n) time. A running total can reduce working storage to O(1) when the original entries do not need to be retained. The list/tuple design scales because the number of entries changes without changing the program structure.

Illustrative data structure

entries = [("F001", 10.0, 3.5, 8.5, "Morning"), ("F002", 8.5, 4.2, 8.7, "Evening")]